pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Debugging

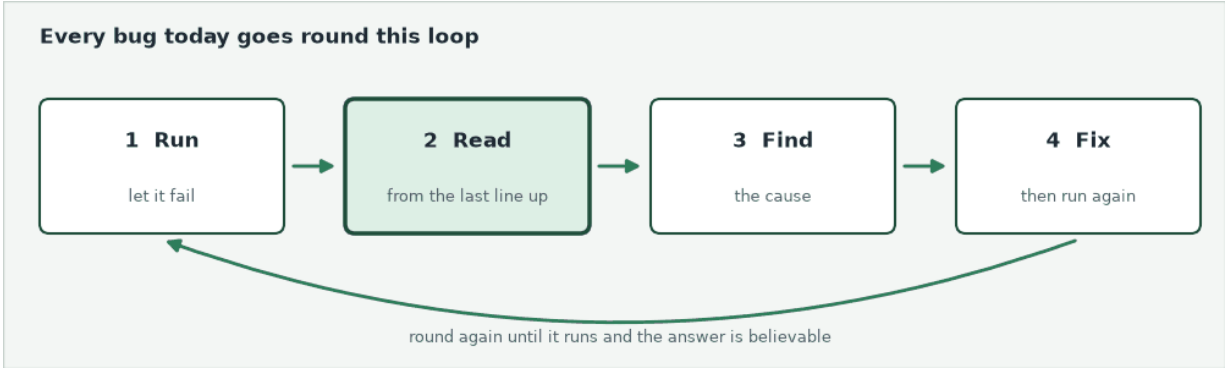
WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's fifth note

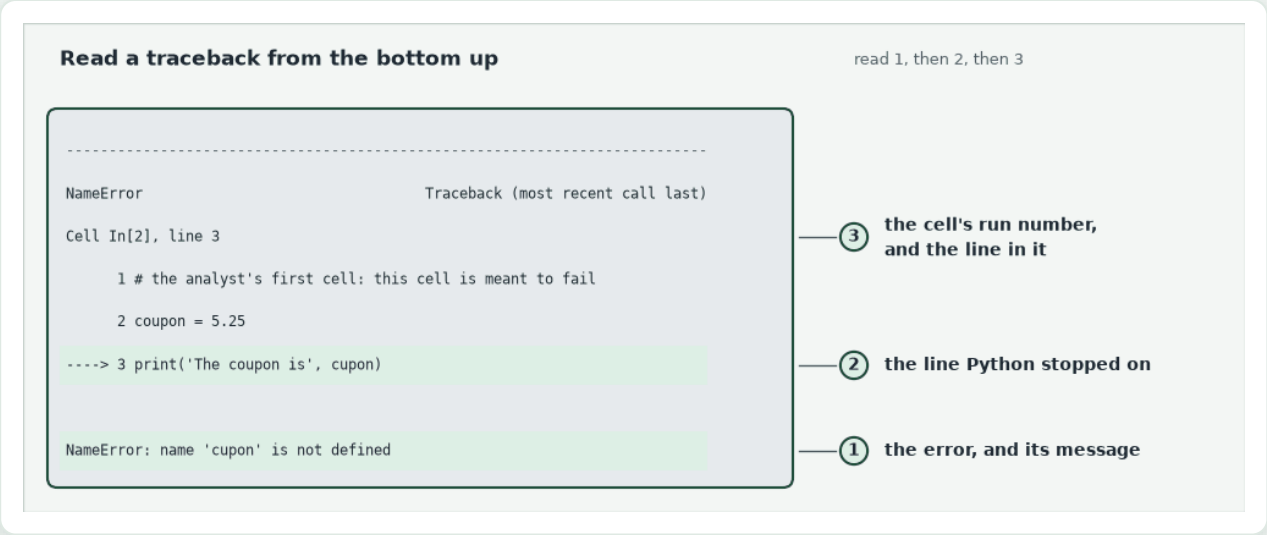
A new analyst wrote this week's code for our book. Half of it stops in red. Worse, some of it runs and prints numbers I do not trust: QVNE's income came out at 7,875,000. Fix every cell, find the wrong numbers, and make the book check its own totals.



A mistake in code is a bug; finding and fixing it is debugging.

```
print(f'The desk holds {len(ticker_list)} bonds and {sum(face_list):,} face')
The desk holds 4 bonds and 3,000,000 face
```

3.1 Read the red block from the last line up



The analyst's cell

```
coupon = 5.25
print('The coupon is', cupon)
NameError: name 'cupon' is not defined
```

Spelled as it was stored

```
coupon = 5.25
print('The coupon is', coupon)
The coupon is 5.25
```

The first actual bug

On September 9, 1947, engineers on Harvard's Mark II found a moth in the machine and taped it into the log book. Engineers already said "bug": Edison used it in 1878. The log book was probably not Grace Hopper's.

Sources: JSTOR Daily; the museum's object record

3.2 Code Python cannot read

```
for ticker in ticker_list
SyntaxError: expected ':'
for ticker in ticker_list:
print(ticker)
IndentationError: expected an indented block after 'for' statement on line 3
if price = 100:
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
new_coupon_list = [5.25, 6.125, 4.75, 7.5, 6.0
SyntaxError: '[' was never closed
```

Excel side by side

Excel refuses too: type =1+ and press Enter, and Excel will not take the formula until it is fixed.

Every bracket, parenthesis, and quote needs its partner.

3.3 Types, names, and methods

```
face = '1500000'
coupon = 5.25
income = face * coupon / 100
TypeError: can't multiply sequence by non-int of type 'float'
```

```
face = int(face)
income = face * coupon / 100
print('The annual coupon income is', income)
The annual coupon income is 78750.0
```

```
whole_points = math.floor(98.5)
print(whole_points)
NameError: name 'math' is not defined
```

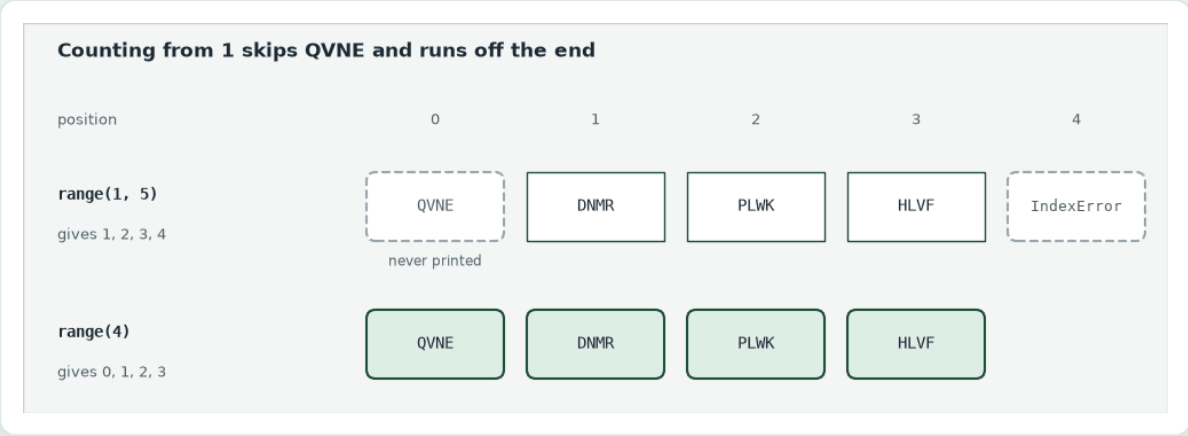
```
import math
whole_points = math.floor(98.5)
print(whole_points)
98
```

```
ticker_list.add('ZXMT')
AttributeError: 'list' object has no attribute 'add'
```

```
ticker_list.append('ZXMT')
print(ticker_list)
ticker_list.pop()
print(ticker_list)
['QVNE', 'DNMR', 'PLWK', 'HLVF', 'ZXMT']
['QVNE', 'DNMR', 'PLWK', 'HLVF']
```

Python never converts for you.

3.4 Counting from one, and keys




```
for i in range(1, 5):
    print(ticker_list[i])
DNMR
PLWK
HLVF
IndexError: list index out of range
for ticker in ticker_list:
```

```
bond = {Ticker: 'QVNE', 'Coupon': 5.25, 'Price': 98.5}
NameError: name 'Ticker' is not defined
print(bond['coupon'])
KeyError: 'coupon'
print(bond.keys())
print(bond['Coupon'])
dict_keys(['Ticker', 'Coupon', 'Price'])
5.25
```

Excel side by side. Excel counts from 1: =INDEX(A1:A4,1) returns the first item. =INDEX(A1:A4,5) on four cells returns #REF!, Excel's IndexError.

3.5 Bad values in the feed

```
coupon_text = '5.25%'
coupon = float(coupon_text)
ValueError: could not convert string to float: '5.25%'
```

```
coupon = float(coupon_text.replace('%', ''))
print(coupon)
5.25
```

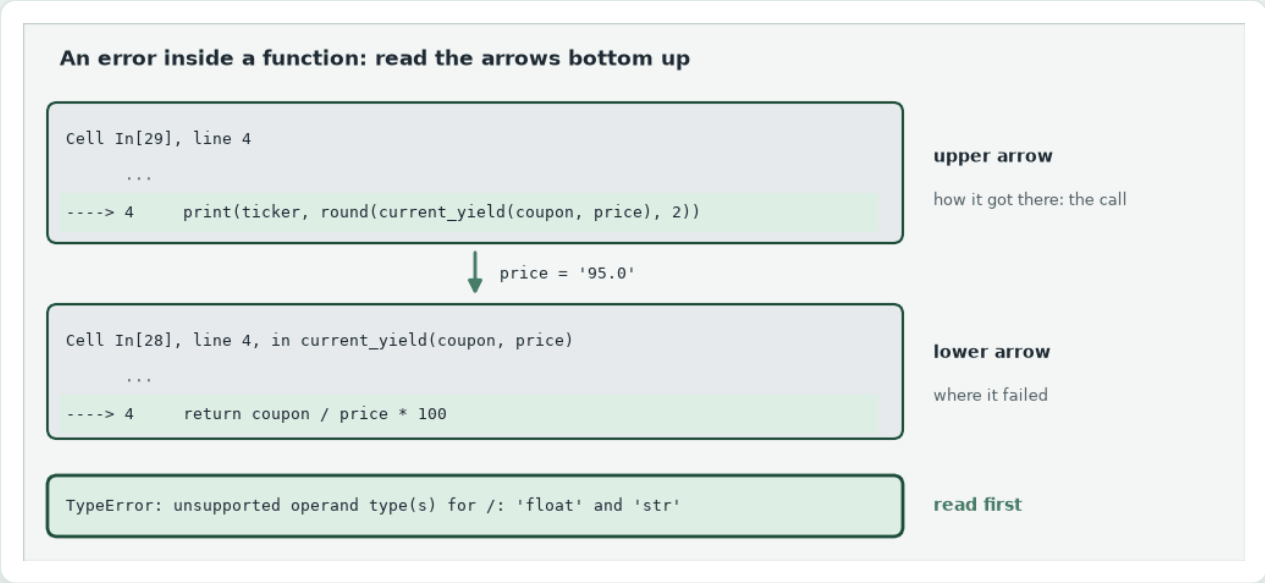
Excel side by side

=VALUE("5.25%") returns 0.0525: Excel reads the percent sign and divides by 100. Our fix gives 5.25, in percent, the way the book stores coupons.

The zero stopped the loop after two bonds: report a bad price, never hide it.

```
for ticker, coupon, price in zip(ticker_list, coupon_list, feed_price_list):
    if price <= 0:
        print(ticker, 'has a bad price:', price)
    else:
        print(ticker, round(coupon / price * 100, 2))
QVNE 5.33
DNMR 5.93
PLWK has a bad price: 0
HLVF 7.43
```

3.6 Two arrows



Lower arrow: where it failed. Upper arrow: the call that brought the bad value in.

Bottom first: a TypeError, a number divided by text.

The function is fine, so the fix goes in the loop.

```
feed_price_list = [98.5, 103.25, '95.0', 101.0]
for ticker, coupon, price in zip(ticker_list, coupon_list, feed_price_list):
    print(ticker, round(current_yield(coupon, price), 2))
QVNE 5.33
DNMR 5.93
TypeError: unsupported operand type(s) for /: 'float' and 'str'
```

Every error so far

Error	Means	First check	Excel's nearest
`SyntaxError, IndentationError`	cannot read the code	colon, bracket, quote, indent	formula refused
`NameError`	no such name	spelling, quotes, import, a cell not run	#NAME?
`TypeError`	wrong type for the operation	type() of each value, as with 'QVNE ' + 5.25 in Part 1	#VALUE!
`ValueError`	right type, wrong content	the text converted, such as '5.25%'	#VALUE!
`IndexError`	no item at that position	the list's length; a range one off	#REF!
`KeyError`	no such key	.keys(): spelling and capitals	#N/A
`AttributeError`	no such attribute on that object	dir(), help()	#NAME?
`ZeroDivisionError`	division by 0	the data	#DIV/0!
`ModuleNotFoundError (Day 0)`	library not installed	spelling, then %pip install	no match
`FileNotFoundError (Day 2)`	nothing at that path	file name, then folder	no match

Excel shows its error in the cell, and in every cell that uses it, and calculates on; Python stops and names the line.

3.7 Wrong numbers, no error

```
face = 1500000
coupon = 5.25
print(f'As the analyst wrote it: {face * coupon:,.2f}')
print(f'Coupon divided by 100:   {face * coupon / 100:,.2f}')
As the analyst wrote it: 7,875,000.00
Coupon divided by 100:   78,750.00
```

Know roughly the answer before you run the code.



The analyst's loop

```
total = 0
for coupon in coupon_list:
    total = coupon
average_coupon = total / len(coupon_list)
print('The average coupon is', average_coupon)
The average coupon is 1.875
```

+= adds each coupon

```
total = 0
for coupon in coupon_list:
    total += coupon
average_coupon = total / len(coupon_list)
print('The average coupon is', average_coupon)
The average coupon is 5.90625
```

Quiet bugs: one list, a taken name, 0.1 plus 0.2



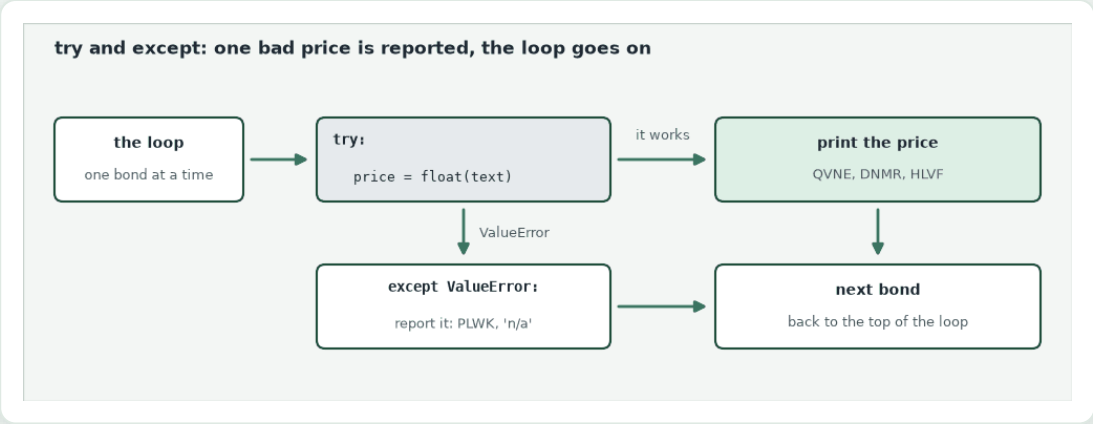

```
price_list[0] = 98.5
what_if_list = price_list.copy()
what_if_list[0] = 99.0
print(f'What-if prices: {what_if_list}')
print(f'Desk prices:    {price_list}')
What-if prices: [99.0, 103.25, 95.0, 101.0]
Desk prices:    [98.5, 103.25, 95.0, 101.0]
```

```
sum = 0
for coupon in coupon_list:
    sum += coupon
print(f'Total of the coupons: {sum}')
average_coupon = sum(coupon_list) / len(coupon_list)
Total of the coupons: 23.625
TypeError: 'float' object is not callable
```

```
total_fee = 0.1 + 0.2
print(total_fee)
print(total_fee == 0.3)
0.30000000000000004
False
```

Excel side by side. Excel stores numbers the same way, yet =0.1+0.2=0.3 returns TRUE in Excel.

3.8 Expect the error, and name it




```
for ticker, text in zip(ticker_list, feed_text_list):
    try:
        price = float(text)
        print(ticker, 'is priced at', price)
    except ValueError:
        print(ticker, 'has a price that is not a number:', text)
QVNE is priced at 98.5
DNMR is priced at 103.25
PLWK has a price that is not a number: n/a
HLVF is priced at 101.0
```

A bare except hides a typo

```
price = flaot(text)
except:
QVNE has a price that is not a number: 98.5
DNMR has a price that is not a number: 103.25
PLWK has a price that is not a number: n/a
HLVF has a price that is not a number: 101.0
```

Excel side by side

=IFERROR(VALUE(A1),"bad price") catches every kind of error, like a bare except:.

Always name the error you expect.

3.9 Checks that stop: the book checks itself

```
assert book_face == 3000000, 'face does not tie'
assert math.isclose(book_cost, 2958750), 'cost does not tie'
assert math.isclose(book_income, 163750), 'income does not tie'
print(f'Book ties: {book_face:,} face, ${book_cost:,.2f} cost, ${book_income:,.2f} income')
Book ties: 3,000,000 face, $2,958,750.00 cost, $163,750.00 income
```

```
faces_with_repeat = face_list + [250000]
total_face = sum(faces_with_repeat)
assert total_face == 3000000, f'expected 3,000,000 face and found {total_face:,}'
AssertionError: expected 3,000,000 face and found 3,250,000
```

Python	Who uses it	Excel's nearest
`assert`	you, on your own totals	a check cell, =IF(SUM(F2:F5)=3000000,"OK","CHECK"), which calculates on
`raise`	a function, on a bad value it is handed	Data Validation (Data tab), which refuses a value as it is typed
`try and except`	the caller, deciding what to do	IFNA, as in 3.8

The note, answered

The note asked	Answer
Fix every cell	18 red blocks read from the last line up: 16 bugs fixed, and 2 checks doing their job
Find the wrong numbers	QVNE's 7,875,000 is 78,750 dollars; the 1.875 average coupon is 5.90625
Make the book check its own totals	three assert lines tie the face, the cost, and the income

A red block no longer says you failed: it gives the type, the message, and the line.

On Day 4 a class makes a Bond that carries its own data and its own calculations.